



ASSIGNMENT 3

Assignment Posted: June 15, 2026

Assignment Due: June 23, 2026, before 11.59 PM

“Libraries organize knowledge. Functional programs organize data. Both succeed through clear structure, simple rules, and meaningful relationships.”

— Inspired by Rich Hickey

Description:

This assignment offers you a deep dive into the world of Clojure. Although any previous experience with the language isn't required, it presents a valuable chance to explore and apply the key concepts of Clojure taught in class. In addition to reinforcing your knowledge of familiar constructs, you'll encounter new ideas and features that will enrich your hands-on experience and enhance your overall programming skills. You'll engage with Clojure's versatile data structures, such as maps, vectors, sets, keywords, and the EDN (Extensible Data Notation) format. By the end of this task, you'll have developed a stronger command of Clojure and its capabilities.

NOTE: Please note that the assignment document may seem lengthy, but the description of the assignment itself is concise and clear. The document primarily consists of valuable hints and advice designed to streamline development of your Clojure program.

Welcome to the Functional Library Information System assignment. Libraries contain large collections of books that must be organized, searched, and analyzed efficiently. In this assignment, you will develop a command-line application that allows users to interact with a collection of books using functional programming principles.

The system will allow users to:

- View books stored in a library collection
- Search for books by title or author
- Filter books by genre or publication year
- Add new books to the collection
- Generate useful library statistics
- Save changes back to an EDN file

All program logic should be expressed using immutable data structures, pure functions whenever possible, and idiomatic Clojure constructs. You will apply functional operations over collections and will work with immutable updates to persistent data structures.

Unlike traditional text-based data files, the library collection will be stored using EDN (Extensible Data Notation), Clojure's native data format. This allows book information to be represented directly as Clojure data structures, simplifying data manipulation and emphasizing the language's strengths.

Data File Format:

The application will load book information from an EDN file.

Example:

```
[{:id "B001"
  :title "The Hobbit"
  :author "J.R.R. Tolkien"
  :genre "Fantasy"
  :year 1937}

{:id "B002"
  :title "Dune"
  :author "Frank Herbert"
  :genre "Science Fiction"
  :year 1965}

{:id "B003"
  :title "Foundation"
  :author "Isaac Asimov"
  :genre "Science Fiction"
  :year 1951}]
```

Each book contains the following fields: id → Unique book identifier, title → Book title, author → Author name, genre → Book genre and year → Publication year.

Implementation Specifications:

Your application must begin by displaying a welcome message and a main menu like the one illustrated in Figure 1. It should offer six core options, assuming perfect user input (correct data types and expected formats).

```
=====
Functional Library Information System
=====

1. View Library Collection
2. Search Library
3. Filter Books
4. Add a New Book
5. Library Statistics
6. Save and Exit

Enter option: █
```

Figure 1. Main menu

Detailed behavior of each menu option is mentioned below with screenshots.

1. View Library Collection

This option displays all books currently stored in the library.

Requirements:

- Display the total number of books.
- Display all books sorted by publication year.
- Display all book information.

2. Search Library

This option allows users to search for books.

Sub-menu:

1. Search by Title

- The user enters a title fragment.
- Display all matching books whose titles contain the entered text.
- The search must be case-insensitive.

2. Search by Author

- The user enters an author name or partial author name.
- Display all matching books.
- The search must be case-insensitive.

3. Filter Books

This option enables users to filter the collection.

Sub-menu:

1. Filter by Genre

- The user enters a genre.
- Display all books whose genre exactly matches the supplied genre.
- The comparison should be case-sensitive.

2. Filter by Publication Year Range

- The user enters:
- Minimum year
- Maximum year

4. Add a New Book

This option allows users to add a new book to the collection.

Prompt the user for:

- Book ID
- Title
- Author
- Genre
- Publication Year

Create a new book map and add it to the collection.

Example:

```
{:id "B010"  
 :title "The Martian"  
 :author "Andy Weir"  
 :genre "Science Fiction"  
 :year 2011}
```

After successfully adding the book, display a confirmation message.

5. Library Statistics

Generate the following statistics:

- **Total Number of Books**
Display the total number of books currently in the collection.
- **Oldest Book**
Display the complete information of the book with the earliest publication year.
- **Newest Book**
Display the complete information of the book with the latest publication year.
- **Number of Unique Genres**
Using a set, determine number of distinct genres represented in the collection.
- **Books per Genre**
Display the number of books belonging to each genre. For example,

Fantasy: 12
Science Fiction: 9
History: 5

6. Save and Exit

When selected:

- Save the current collection back to the EDN file.
- Display a confirmation message.
- Exit the application.

Required Clojure Features:

Your solution must demonstrate the use of:

- Maps
- Vectors
- Keywords
- Sets
- Higher-order functions
 - map
 - filter
 - reduce

- Immutable updates
- File I/O
- EDN reading and writing
- User-defined functions
- Menu-driven program design

Assumptions:

You may assume:

- The user always enters valid menu options.
- The EDN file is correctly formatted.
- Publication years are valid integers.
- Book IDs are unique.
- No error handling is required for invalid input.

MANDATORY DATA STRUCTURES AND FUNCTIONAL PROGRAMMING USAGE:

1. Maps:
 - Each book must be represented as a map containing the following keys:
 - ✓ :id
 - ✓ :title
 - ✓ :author
 - ✓ :genre
 - ✓ :year
 - All book information must be stored and manipulated using maps.
2. Vectors:
 - The entire library collection must be stored in a vector of book maps.
 - Vector operations should be used when traversing, filtering, sorting, and updating the collection.
3. Sets:
 - Sets must be used to determine the number of distinct genres present in the library collection.
 - Appropriate set operations should be used whenever unique values are required.
4. EDN:
 - The library collection must be loaded from an EDN file at program startup.
 - The updated collection must be written back to an EDN file when the user selects Save and Exit.
 - Students are expected to use the clojure.edn library for reading and writing data.
5. Searching and Filtering:
 - The Search Library and Filter Books features must be implemented using higher-order functions.
 - The filter function should be used to identify books that satisfy user-specified criteria.
6. Sorting:
 - Books displayed through the View Library Collection option must be sorted by publication year.

- The sort-by function should be used to perform sorting operations.
- 7. Statistics and Data Analysis:
 - Library statistics must be computed using functional operations over collections.
 - Functions such as group-by, frequencies, map, and reduce should be used where appropriate to generate statistical summaries.
- 8. Immutable Updates:
 - Adding a new book must not modify an existing collection directly.
 - A new collection containing the additional book should be created using immutable collection operations such as conj.
- 9. Functional Programming Requirements:
 - Core application logic must be implemented using pure functions where possible.
 - Explicit state mutation using atom, ref, agent, mutable Java collections, or similar mechanisms is not permitted.
 - Input/output operations should be confined to dedicated functions responsible for user interaction and file processing.
 - Students are expected to make meaningful use of higher-order functions such as:
 - ✓ map
 - ✓ filter
 - ✓ reduce
 - ✓ group-by
 - ✓ sort-by
 - ✓ frequencies

Failure to demonstrate meaningful use of the required data structures and functional programming techniques may result in deductions, even if the program produces correct output.

Submission:

The assignment must be submitted only through Moodle. No other form of submission will be considered. Please create a zip file containing your Clojure code and a readme file (txt). The zip file should be named Assignment#3_YourStudentID. In the readme document, the features and functionality of the application, and anything else you want grader to know.

Evaluation Procedure

Please note that the markers will be using a standard Linux system, likely the same docker installation that most of you should be using. Your code MUST compile and run from the Linux command line. If you are using an IDE (e.g., IntelliJ or VS Code), rather than a simple editor, you will want to test execution from the command line, since IDEs often create their own paths, folders, environment variables that might prevent the code from running on the command line.

To evaluate the submissions, the marker will simply create a test folder and add a sample input data file (library.edn) plus your source files. Your code will be compiled and run and each of the menu options will be tested. Every student will be graded the same way. The markers will be given a simple spreadsheet that lists the criteria described above.

There are no mystery requirements. Please note that it is better to have a working version of a slightly restricted program than a non-working version of something that tries to do everything (e.g., you might be able to display the entries, but you can't format it, or you can display the existing entries, but you can't add any). Hence, make sure that your code compiles and runs. It is virtually impossible to grade an assignment that does not run.

THIS ASSIGNMENT MUST BE SUBMITTED INDIVIDUALLY.

IMPORTANT!!!

Markers will receive a straightforward spreadsheet containing the specified criteria outlined earlier. There are no undisclosed or hidden requirements. It is emphasized that having a functional version of a somewhat limited program is preferable to having a non-functional version attempting to cover every aspect. Therefore, prioritize ensuring that your code can be successfully compiled and executed. Evaluating an assignment that doesn't run at all is virtually impossible.

Grading Rubric

Component	Description	Marks
View Library Collection	Correctly displays all books, total count, and sorted output	10 pts
Search Library	Correct implementation of title and author searches (case-insensitive)	10 pts
Filter Books	Correct implementation of genre and publication year filtering	10 pts
Add a New Book	Correctly creates and adds new books to the collection using immutable updates	10 pts
Library Statistics	Correct computation of total books, oldest/newest books, unique genres, and books per genre	15 pts
EDN File Processing	Correct loading and saving of data using EDN and clojure.edn	10 pts
Functional Programming Techniques	Meaningful use of immutable data structures, pure functions, and higher-order functions (map, filter, reduce, group-by, sort-by, etc.)	15 pts
Required Data Structures	Appropriate use of maps, vectors, keywords, and sets throughout the solution	15 pts
Program Organization	Modular design, clear function decomposition, meaningful naming, and menu structure	3 pts
Readability and Documentation	Consistent formatting, indentation, and appropriate comments	2 pts
Total		100 pts

Sample Run:

```
=====
Functional Library Information System
=====
```

1. View Library Collection
2. Search Library
3. Filter Books
4. Add a New Book
5. Library Statistics
6. Save and Exit

Enter option: 1

Total Books: 5

```
-----
ID:      B005
Title:   Pride and Prejudice
Author:  Jane Austen
Genre:   Classic
Year:    1813
-----
ID:      B001
Title:   The Hobbit
Author:  J.R.R. Tolkien
Genre:   Fantasy
Year:    1937
-----
ID:      B003
Title:   Foundation
Author:  Isaac Asimov
Genre:   Science Fiction
Year:    1951
-----
ID:      B002
Title:   Dune
Author:  Frank Herbert
Genre:   Science Fiction
Year:    1965
-----
ID:      B004
Title:   The Martian
Author:  Andy Weir
Genre:   Science Fiction
Year:    2011
-----
```

```
=====
```


Functional Library Information System
=====

1. View Library Collection
2. Search Library
3. Filter Books
4. Add a New Book
5. Library Statistics
6. Save and Exit

Enter option: 2

1. Search by Title
2. Search by Author

Enter option: 1

Enter title fragment: du

Total Books: 1

ID: B002
Title: Dune
Author: Frank Herbert
Genre: Science Fiction
Year: 1965

=====

Functional Library Information System
=====

1. View Library Collection
2. Search Library
3. Filter Books
4. Add a New Book
5. Library Statistics
6. Save and Exit

Enter option: 2

1. Search by Title
2. Search by Author

Enter option: 2

Enter author name: en

Total Books: 2

ID: B005

Title: Pride and Prejudice
Author: Jane Austen
Genre: Classic
Year: 1813

ID: B001
Title: The Hobbit
Author: J.R.R. Tolkien
Genre: Fantasy
Year: 1937

=====
Functional Library Information System
=====

1. View Library Collection
2. Search Library
3. Filter Books
4. Add a New Book
5. Library Statistics
6. Save and Exit

Enter option: 3

1. Filter by Genre
2. Filter by Publication Year Range

Enter option: 1

Enter genre: fantasy

No books found.

=====
Functional Library Information System
=====

1. View Library Collection
2. Search Library
3. Filter Books
4. Add a New Book
5. Library Statistics
6. Save and Exit

Enter option: 3

1. Filter by Genre
2. Filter by Publication Year Range

Enter option: 1

Enter genre: Fantasy

Total Books: 1

ID: B001
Title: The Hobbit
Author: J.R.R. Tolkien
Genre: Fantasy
Year: 1937

=====
Functional Library Information System
=====

1. View Library Collection
2. Search Library
3. Filter Books
4. Add a New Book
5. Library Statistics
6. Save and Exit

Enter option: 3

1. Filter by Genre
2. Filter by Publication Year Range

Enter option: 2

Enter minimum year: 1800
Enter maximum year: 1952

Total Books: 3

ID: B005
Title: Pride and Prejudice
Author: Jane Austen
Genre: Classic
Year: 1813

ID: B001
Title: The Hobbit
Author: J.R.R. Tolkien
Genre: Fantasy
Year: 1937

ID: B003
Title: Foundation
Author: Isaac Asimov
Genre: Science Fiction
Year: 1951

```
=====
Functional Library Information System
=====
```

1. View Library Collection
2. Search Library
3. Filter Books
4. Add a New Book
5. Library Statistics
6. Save and Exit

Enter option: 4

Enter Book ID: B006
Enter Title: Brave New World
Enter Author: Aldous Huxley
Enter Genre: Dystopian
Enter Publication Year: 1949

Book added successfully.

```
=====
Functional Library Information System
=====
```

1. View Library Collection
2. Search Library
3. Filter Books
4. Add a New Book
5. Library Statistics
6. Save and Exit

Enter option: 5

===== Library Statistics =====

Total Books: 6

Oldest Book:

```
-----
ID:      B005
Title:   Pride and Prejudice
Author:  Jane Austen
Genre:   Classic
Year:    1813
```

Newest Book:

```
-----
ID:      B004
```

Title: The Martian
Author: Andy Weir
Genre: Science Fiction
Year: 2011

Unique Genres: 4

Books Per Genre:
Fantasy : 1
Science Fiction : 3
Classic : 1
Dystopian : 1

=====
Functional Library Information System
=====

1. View Library Collection
2. Search Library
3. Filter Books
4. Add a New Book
5. Library Statistics
6. Save and Exit

Enter option: 1

Total Books: 6

ID: B005
Title: Pride and Prejudice
Author: Jane Austen
Genre: Classic
Year: 1813

ID: B001
Title: The Hobbit
Author: J.R.R. Tolkien
Genre: Fantasy
Year: 1937

ID: B006
Title: Brave New World
Author: Aldous Huxley
Genre: Dystopian
Year: 1949

ID: B003
Title: Foundation
Author: Isaac Asimov

Genre: Science Fiction
Year: 1951

ID: B002
Title: Dune
Author: Frank Herbert
Genre: Science Fiction
Year: 1965

ID: B004
Title: The Martian
Author: Andy Weir
Genre: Science Fiction
Year: 2011

=====
Functional Library Information System
=====

1. View Library Collection
2. Search Library
3. Filter Books
4. Add a New Book
5. Library Statistics
6. Save and Exit

Enter option: 6

Library collection saved successfully.
Exiting program...